AI Assistant Coding

Assignment 12.1

Name : Vyshnavi Vallala **HT. No :** 2303A52429 **Batch:** 32

Task Description #1 (Sorting – Merge Sort Implementation)

• **Task:** Use AI to generate a Python program that implements the Merge Sort algorithm**.**

• **Instructions:**

- Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
- Ask AI to include time complexity and space complexity in the function docstring.
- Verify the generated code with test cases.

Code:

```
def merge_sort(arr):
    """
    Sort a list in ascending order using Merge Sort.

    Time Complexity:
        - Best case: O(n log n)
        - Average case: O(n log n)
        - Worst case: O(n log n)

    Space Complexity:
        - O(n) auxiliary space for temporary sublists created during merging.

    Args:
        arr (list): The input list of comparable elements.

    Returns:
        list: A new sorted list in ascending order.
    """
    if len(arr) <= 1:
        return arr[:]

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])

    return _merge(left, right)


def _merge(left, right):
    merged = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged


if __name__== "__main__":
    test_cases = [
        [],
        [1],
        [5, 2, 9, 1, 5, 6],
        [3, -1, 0, -7, 8, 2],
        [10, 9, 8, 7, 6, 5],
        [4, 4, 4, 4],
    ]

    for case in test_cases:
        print(f"Original: {case} -> Sorted: {merge_sort(case)}")
```

Output:

```
Original: [] -> Sorted: []
Original: [1] -> Sorted: [1]
Original: [5, 2, 9, 1, 5, 6] -> Sorted: [1, 2, 5, 5, 6, 9]
Original: [3, -1, 0, -7, 8, 2] -> Sorted: [-7, -1, 0, 2, 3, 8]
Original: [10, 9, 8, 7, 6, 5] -> Sorted: [5, 6, 7, 8, 9, 10]
Original: [4, 4, 4, 4] -> Sorted: [4, 4, 4, 4]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• **Task:** Use AI to create a binary search function that finds a target element in a sorted list.

• **Instructions:**

- Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
- Include docstrings explaining best, average, and worst-case complexities.
- Test with various inputs**.**

Code:

```
def binary_search(arr, target):
    """
    Find the index of target in a sorted list using Binary Search.

    Complexity:
        - Best case: O(1) when the middle element is the target.
        - Average case: O(log n)
        - Worst case: O(log n)
        - Space complexity: O(1) for iterative implementation.

    Args:
        arr (list): Sorted list of comparable elements in ascending order.
        target: Element to search for.

    Returns:
        int: Index of target if found, otherwise -1.
    """
    left = 0
    right = len(arr) - 1

    # Narrow the search range by comparing against the midpoint each
iteration.
    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            left = mid + 1
```

```
        else:
            right = mid - 1

    return -1


if __name__ == "__main__":
    test_cases = [
        ([1, 3, 5, 7, 9], 5),
        ([1, 3, 5, 7, 9], 1),
        ([1, 3, 5, 7, 9], 9),
        ([1, 3, 5, 7, 9], 4),
        ([], 10),
        ([2], 2),
        ([2], 3),
        ([-10, -3, 0, 4, 8, 12], -3),
    ]

    for arr, target in test_cases:
        result = binary_search(arr, target)
        print(f"arr={arr}, target={target} -> index={result}")
```

Output:

```
arr=[1, 3, 5, 7, 9], target=5 -> index=2
arr=[1, 3, 5, 7, 9], target=1 -> index=0
arr=[1, 3, 5, 7, 9], target=9 -> index=4
arr=[1, 3, 5, 7, 9], target=4 -> index=-1
arr=[], target=10 -> index=-1
arr=[2], target=2 -> index=0
arr=[2], target=3 -> index=-1
arr=[-10, -3, 0, 4, 8, 12], target=-3 -> index=1
```

Task Description #3 (Real-Time Application – Inventory Management System)

• **Scenario:** A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

• **Task:**

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Code:

```
"""Task #3: Real-Time Inventory Management System.

This module demonstrates efficient algorithms for:
1) Fast product lookup by ID and name.
2) Sorting products by price or stock quantity.
"""

from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional


@dataclass(frozen=True)
class Product:
    """Represents one inventory item."""

    product_id: str
    name: str
    price: float
    quantity: int


def build_indexes(products: Iterable[Product]) -> tuple[Dict[str, Product],
Dict[str, List[Product]]]:
    """Build hash-based indexes for fast exact-match searches.

    Algorithms used:
    - Hash table (Python dict) for ID lookup: average O(1)
    - Hash table (Python dict) for name lookup: average O(1)

    Args:
        products: Iterable of Product records.

    Returns:
        Tuple:
            - products_by_id: maps product_id -> Product
            - products_by_name: maps normalized name -> list[Product]
    """
    products_by_id: Dict[str, Product] = {}
    products_by_name: Dict[str, List[Product]] = {}
```

```
    for product in products:
        products_by_id[product.product_id] = product

        normalized_name = product.name.strip().lower()
        products_by_name.setdefault(normalized_name, []).append(product)

    return products_by_id, products_by_name


def search_by_id(products_by_id: Dict[str, Product], product_id: str) ->
Optional[Product]:
    """Return a product by ID using hash lookup.

    Complexity:
    - Best/Average: O(1)
    - Worst: O(n) in pathological hash-collision scenarios
    - Space: O(1) extra per lookup
    """
    return products_by_id.get(product_id)


def search_by_name(products_by_name: Dict[str, List[Product]], name: str) ->
List[Product]:
    """Return products that exactly match a name (case-insensitive).

    Complexity:
    - Best/Average: O(1) for dictionary lookup
    - Worst: O(n) with heavy collisions
    - Space: O(k) for returned results, where k is number of matches
    """
    normalized_name = name.strip().lower()
    return products_by_name.get(normalized_name, [])


def sort_inventory(products: Iterable[Product], by: str, descending: bool =
False) -> List[Product]:
    """Sort products by price or quantity using Python's Timsort.

    Timsort is a hybrid stable sorting algorithm used by Python's sorted().
    It performs very well on real-world data and partially ordered datasets.

    Complexity (Timsort):
    - Best: O(n) on nearly sorted data
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    valid_keys = {"price", "quantity"}
    if by not in valid_keys:
```

```
        raise ValueError(f"Invalid sort field '{by}'. Choose one of:
{sorted(valid_keys)}")

    if by == "price":
        return sorted(products, key=lambda p: p.price, reverse=descending)

    return sorted(products, key=lambda p: p.quantity, reverse=descending)


if __name__ == "__main__":
    sample_inventory = [
        Product("P1001", "Keyboard", 1200.0, 35),
        Product("P1002", "Mouse", 600.0, 70),
        Product("P1003", "Monitor", 8500.0, 18),
        Product("P1004", "Laptop", 56000.0, 12),
        Product("P1005", "Keyboard", 1500.0, 22),
    ]

    by_id, by_name = build_indexes(sample_inventory)

    print("Search by ID P1003:", search_by_id(by_id, "P1003"))
    print("Search by ID P9999:", search_by_id(by_id, "P9999"))
    print("Search by name 'Keyboard':", search_by_name(by_name, "Keyboard"))

    print("\nSorted by price (low to high):")
    for item in sort_inventory(sample_inventory, by="price"):
        print(item)

    print("\nSorted by quantity (high to low):")
    for item in sort_inventory(sample_inventory, by="quantity",
descending=True):
        print(item)
```

Output:

```
Search by ID P1003: Product(product_id='P1003', name='Monitor', price=8500.0,
quantity=18)
Search by ID P9999: None
Search by name 'Keyboard': [Product(product_id='P1001', name='Keyboard',
price=1200.0, quantity=35), Product(product_id='P1005', name='Keyboard',
price=1500.0, quantity=22)]

Sorted by price (low to high):
Product(product_id='P1002', name='Mouse', price=600.0, quantity=70)
Product(product_id='P1001', name='Keyboard', price=1200.0, quantity=35)
Product(product_id='P1005', name='Keyboard', price=1500.0, quantity=22)
Product(product_id='P1003', name='Monitor', price=8500.0, quantity=18)
Product(product_id='P1004', name='Laptop', price=56000.0, quantity=12)
```

```
Sorted by quantity (high to low):
Product(product_id='P1002', name='Mouse', price=600.0, quantity=70)
Product(product_id='P1001', name='Keyboard', price=1200.0, quantity=35)
Product(product_id='P1005', name='Keyboard', price=1500.0, quantity=22)
Product(product_id='P1003', name='Monitor', price=8500.0, quantity=18)
Product(product_id='P1004', name='Laptop', price=56000.0, quantity=12)
```

Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.

2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms in terms of efficiency and suitability.
- Implement the recommended algorithms in Python.

Code:

```
"""Task #4: Smart Hospital Patient Management System.

This script provides:
1) Recommended algorithms for searching and sorting patient records.
2) Justification for why they fit hospital-scale data (thousands of records).
3) Python implementations for fast lookup and prioritization sorting.
"""

from dataclasses import dataclass
from datetime import date
from typing import Dict, Iterable, List, Optional, Tuple


# Lower rank means higher treatment priority.
SEVERITY_RANK = {
    "critical": 1,
```

```
    "high": 2,
    "medium": 3,
    "low": 4,
}


@dataclass(frozen=True)
class Patient:
    """Represents one patient record in the hospital system."""

    patient_id: str
    name: str
    severity_level: str
    admission_date: date
    bill_amount: float


def algorithm_recommendations() -> List[Tuple[str, str, str]]:
    """Return operation -> algorithm -> justification mapping.

    The recommendations assume:
    - Dataset size: thousands of patients
    - Update frequency: frequent additions/updates through the day
    - Performance need: near real-time lookup for staff workflows
    """
    return [
        (
            "Search by patient ID",
            "Hash table (dict index)",
            "Average O(1) lookup; ideal for unique IDs and frequent queries.",
        ),
        (
            "Search by patient name (exact)",
            "Hash table (dict of normalized name -> patients)",
            "Average O(1) name lookup; supports multiple patients with same
name.",
        ),
        (
            "Sort by severity level",
            "Timsort (Python sorted) with severity rank key",
            "Stable O(n log n) worst-case, O(n) on near-sorted data; robust
for repeated reporting.",
        ),
        (
            "Sort by bill amount",
            "Timsort (Python sorted) by numeric key",
            "Efficient and stable for billing summaries; production-grade
implementation in Python.",
        ),
```

```
    ]


def build_patient_indexes(
    patients: Iterable[Patient],
) -> Tuple[Dict[str, Patient], Dict[str, List[Patient]]]:
    """Build indexes for fast patient lookup.

    Complexity:
    - Build time: O(n)
    - Extra space: O(n)
    """
    by_id: Dict[str, Patient] = {}
    by_name: Dict[str, List[Patient]] = {}

    for patient in patients:
        by_id[patient.patient_id] = patient
        normalized_name = patient.name.strip().lower()
        by_name.setdefault(normalized_name, []).append(patient)

    return by_id, by_name


def search_by_patient_id(by_id: Dict[str, Patient], patient_id: str) ->
Optional[Patient]:
    """Search one patient by ID using hash index.

    Complexity:
    - Best/Average: O(1)
    - Worst: O(n) with pathological collisions
    """
    return by_id.get(patient_id)


def search_by_patient_name(by_name: Dict[str, List[Patient]], name: str) ->
List[Patient]:
    """Search patients by exact name (case-insensitive) using hash index.

    Complexity:
    - Best/Average: O(1) lookup + O(k) output size
    - Worst: O(n)
    """
    return by_name.get(name.strip().lower(), [])


def sort_patients_by_severity(
    patients: Iterable[Patient],
    descending: bool = False,
) -> List[Patient]:
```

```
    """Sort patients by severity priority.

    Critical is highest priority by default.

    Complexity (Timsort):
    - Best: O(n)
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    return sorted(
        patients,
        key=lambda p: SEVERITY_RANK.get(p.severity_level.strip().lower(),
999),
        reverse=descending,
    )


def sort_patients_by_bill(patients: Iterable[Patient], descending: bool =
True) -> List[Patient]:
    """Sort patients by bill amount.

    Complexity (Timsort):
    - Best: O(n)
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    return sorted(patients, key=lambda p: p.bill_amount, reverse=descending)


def print_recommendation_table(rows: List[Tuple[str, str, str]]) -> None:
    """Print operation-algorithm-justification as a readable table."""
    headers = ("Operation", "Recommended Algorithm", "Justification")
    col_widths = [len(h) for h in headers]

    for row in rows:
        for i, value in enumerate(row):
            col_widths[i] = max(col_widths[i], len(value))

    separator = "-+-".join("-" * width for width in col_widths)
    header_line = " | ".join(headers[i].ljust(col_widths[i]) for i in
range(3))

    print(header_line)
    print(separator)
    for row in rows:
        print(" | ".join(row[i].ljust(col_widths[i]) for i in range(3)))


if __name__ == "__main__":
```

```
    sample_patients = [
        Patient("PT1001", "Asha", "critical", date(2026, 3, 20), 125000.0),
        Patient("PT1002", "Ravi", "medium", date(2026, 3, 21), 32000.0),
        Patient("PT1003", "Fatima", "high", date(2026, 3, 19), 58000.0),
        Patient("PT1004", "Ravi", "low", date(2026, 3, 24), 18000.0),
        Patient("PT1005", "John", "critical", date(2026, 3, 18), 210000.0),
    ]

    print("\nAlgorithm Recommendation Table:\n")
    print_recommendation_table(algorithm_recommendations())

    by_id_index, by_name_index = build_patient_indexes(sample_patients)

    print("\nSearch Tests:")
    print("ID PT1003 ->", search_by_patient_id(by_id_index, "PT1003"))
    print("ID PT9999 ->", search_by_patient_id(by_id_index, "PT9999"))
    print("Name 'Ravi' ->", search_by_patient_name(by_name_index, "Ravi"))

    print("\nSorted by Severity (highest priority first):")
    for patient in sort_patients_by_severity(sample_patients):
        print(patient)

    print("\nSorted by Bill Amount (highest first):")
    for patient in sort_patients_by_bill(sample_patients, descending=True):
        print(patient)
```

Output:

```
Algorithm Recommendation Table:

Operation                       | Recommended
Algorithm                             | Justification
--------------------------------------
------+
------------------
Search by patient ID            | Hash table (dict
index)                           | Average O(1) lookup; ideal for unique IDs
and frequent queries.
Search by patient name (exact) | Hash table (dict of normalized name ->
patients) | Average O(1) name lookup; supports multiple patients with same
name.
Sort by severity level          | Timsort (Python sorted) with severity rank
key    | Stable O(n log n) worst-case, O(n) on near-sorted data; robust for
repeated reporting.
Sort by bill amount             | Timsort (Python sorted) by numeric
key            | Efficient and stable for billing summaries; production-grade
implementation in Python.
```

```
Search Tests:
ID PT1003 -> Patient(patient_id='PT1003', name='Fatima',
severity_level='high', admission_date=datetime.date(2026, 3, 19),
bill_amount=58000.0)
ID PT9999 -> None
Name 'Ravi' -> [Patient(patient_id='PT1002', name='Ravi',
severity_level='medium', admission_date=datetime.date(2026, 3, 21),
bill_amount=32000.0), Patient(patient_id='PT1004', name='Ravi',
severity_level='low', admission_date=datetime.date(2026, 3, 24),
bill_amount=18000.0)]

Sorted by Severity (highest priority first):
Patient(patient_id='PT1001', name='Asha', severity_level='critical',
admission_date=datetime.date(2026, 3, 20), bill_amount=125000.0)
Patient(patient_id='PT1005', name='John', severity_level='critical',
admission_date=datetime.date(2026, 3, 18), bill_amount=210000.0)
Patient(patient_id='PT1003', name='Fatima', severity_level='high',
admission_date=datetime.date(2026, 3, 19), bill_amount=58000.0)
Patient(patient_id='PT1002', name='Ravi', severity_level='medium',
admission_date=datetime.date(2026, 3, 21), bill_amount=32000.0)
Patient(patient_id='PT1004', name='Ravi', severity_level='low',
admission_date=datetime.date(2026, 3, 24), bill_amount=18000.0)

Sorted by Bill Amount (highest first):
Patient(patient_id='PT1005', name='John', severity_level='critical',
admission_date=datetime.date(2026, 3, 18), bill_amount=210000.0)
Patient(patient_id='PT1001', name='Asha', severity_level='critical',
admission_date=datetime.date(2026, 3, 20), bill_amount=125000.0)
Patient(patient_id='PT1003', name='Fatima', severity_level='high',
admission_date=datetime.date(2026, 3, 19), bill_amount=58000.0)
Patient(patient_id='PT1002', name='Ravi', severity_level='medium',
admission_date=datetime.date(2026, 3, 21), bill_amount=32000.0)
Patient(patient_id='PT1004', name='Ravi', severity_level='low',
admission_date=datetime.date(2026, 3, 24), bill_amount=18000.0)
```

Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.

2. Sort students based on marks to generate rank lists.

Student Task

- Identify efficient searching and sorting algorithms using AI assistance.
- Justify the choice of algorithms.
- Implement the algorithms in Python.

Code:

```
"""Task #5: University Examination Result Processing System.

This module provides:
1) Fast search of student results using roll number.
2) Sorting by marks to generate rank lists.
3) A recommendation table that maps operations to algorithms and
justification.
"""

from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional, Tuple


@dataclass(frozen=True)
class StudentResult:
    """Represents one student exam result record."""

    roll_number: str
    name: str
    subject: str
    marks: float


def algorithm_recommendations() -> List[Tuple[str, str, str]]:
    """Return operation -> recommended algorithm -> justification.

    Assumptions:
    - Data size: thousands of student records
    - Query pattern: frequent roll-number lookups
    - Output pattern: repeated rank-list generation after updates/imports
    """
    return [
        (
            "Search by roll number",
            "Hash table (dict index)",
            "Average O(1) lookup for unique roll numbers; ideal for frequent
direct queries.",
        ),
```

```
        (
            "Generate rank list by marks",
            "Timsort (Python sorted) with marks key",
            "Stable O(n log n) worst-case and very efficient on real-world
data.",
        ),
    ]


def build_roll_index(results: Iterable[StudentResult]) -> Dict[str,
StudentResult]:
    """Build a dictionary index for roll-number lookup.

    Complexity:
    - Build time: O(n)
    - Extra space: O(n)
    """
    index: Dict[str, StudentResult] = {}
    for record in results:
        index[record.roll_number] = record
    return index


def search_by_roll_number(index: Dict[str, StudentResult], roll_number: str) -
> Optional[StudentResult]:
    """Search a student record by roll number.

    Complexity:
    - Best/Average: O(1)
    - Worst: O(n) with pathological collisions
    """
    return index.get(roll_number)


def sort_by_marks(results: Iterable[StudentResult], descending: bool = True) -
> List[StudentResult]:
    """Sort records by marks for rank generation.

    Complexity (Timsort):
    - Best: O(n)
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    return sorted(results, key=lambda r: r.marks, reverse=descending)


def generate_rank_list(results: Iterable[StudentResult]) -> List[Tuple[int,
StudentResult]]:
    """Generate a rank list where highest marks get best rank.
```

```
    Ties receive the same rank, and the next rank skips accordingly
    (standard competition ranking: 1, 2, 2, 4).
    """
    sorted_results = sort_by_marks(results, descending=True)

    ranked: List[Tuple[int, StudentResult]] = []
    current_rank = 0
    previous_marks: Optional[float] = None

    for position, record in enumerate(sorted_results, start=1):
        if previous_marks is None or record.marks < previous_marks:
            current_rank = position
        ranked.append((current_rank, record))
        previous_marks = record.marks

    return ranked


def print_recommendation_table(rows: List[Tuple[str, str, str]]) -> None:
    """Print operation-algorithm-justification in a readable table."""
    headers = ("Operation", "Recommended Algorithm", "Justification")
    col_widths = [len(h) for h in headers]

    for row in rows:
        for i, value in enumerate(row):
            col_widths[i] = max(col_widths[i], len(value))

    line = "-+-".join("-" * w for w in col_widths)
    print(" | ".join(headers[i].ljust(col_widths[i]) for i in range(3)))
    print(line)
    for row in rows:
        print(" | ".join(row[i].ljust(col_widths[i]) for i in range(3)))


if __name__ == "__main__":
    sample_results = [
        StudentResult("U001", "Asha", "Math", 91),
        StudentResult("U002", "Ravi", "Math", 86),
        StudentResult("U003", "Fatima", "Math", 91),
        StudentResult("U004", "John", "Math", 74),
        StudentResult("U005", "Mina", "Math", 98),
    ]

    print("\nAlgorithm Recommendation Table:\n")
    print_recommendation_table(algorithm_recommendations())

    roll_index = build_roll_index(sample_results)
    print("\nSearch Tests:")
```

```
    print("U003 ->", search_by_roll_number(roll_index, "U003"))
    print("U999 ->", search_by_roll_number(roll_index, "U999"))

    print("\nRank List:")
    for rank, record in generate_rank_list(sample_results):
        print(f"Rank {rank}: {record.roll_number} | {record.name} |
Marks={record.marks}")
```

Output:

```
Algorithm Recommendation Table:

Operation                    | Recommended Algorithm                  |
Justification
------------------------------
--------------------------------------------------------------------------------
Search by roll number        | Hash table (dict index)                | Average
O(1) lookup for unique roll numbers; ideal for frequent direct queries.
Generate rank list by marks | Timsort (Python sorted) with marks key | Stable
O(n log n) worst-case and very efficient on real-world data.

Search Tests:
U003 -> StudentResult(roll_number='U003', name='Fatima', subject='Math',
marks=91)
U999 -> None

Rank List:
Rank 1: U005 | Mina | Marks=98
Rank 2: U001 | Asha | Marks=91
Rank 2: U003 | Fatima | Marks=91
Rank 4: U002 | Ravi | Marks=86
Rank 5: U004 | John | Marks=74
```

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.

2. Sort orders based on delivery time or price.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the algorithm selection.
- Implement searching and sorting modules in Python.

Code:

```
"""Task #6: Online Food Delivery Platform.

This module provides:
1) Efficient order lookup by order ID.
2) Sorting orders by delivery time or price.
3) A recommendation table mapping operations to algorithms with justification.
"""

from dataclasses import dataclass
from datetime import datetime
from typing import Dict, Iterable, List, Optional, Tuple


@dataclass(frozen=True)
class Order:
    """Represents one food delivery order."""

    order_id: str
    restaurant_name: str
    delivery_time: datetime
    price: float
    order_status: str


def algorithm_recommendations() -> List[Tuple[str, str, str]]:
    """Return operation -> recommended algorithm -> justification.

    Assumptions:
    - Data size: thousands of orders.
    - Workload: frequent direct order-ID checks from users/support staff.
    - Output: repeated sorting for dashboards and operations.
    """
    return [
        (
            "Search by order ID",
            "Hash table (dict index)",
            "Average O(1) lookup for unique IDs; best fit for high-frequency
real-time queries.",
        ),
        (
            "Sort by delivery time",
            "Timsort (Python sorted) with datetime key",
```

```
            "Stable O(n log n) worst-case and fast on real-world partially
ordered data.",
        ),
        (
            "Sort by price",
            "Timsort (Python sorted) with numeric key",
            "Production-grade stable sorting for billing and analytics
views.",
        ),
    ]


def build_order_index(orders: Iterable[Order]) -> Dict[str, Order]:
    """Build order ID index for fast retrieval.

    Complexity:
    - Build time: O(n)
    - Extra space: O(n)
    """
    index: Dict[str, Order] = {}
    for order in orders:
        index[order.order_id] = order
    return index


def search_order_by_id(index: Dict[str, Order], order_id: str) ->
Optional[Order]:
    """Find an order by ID.

    Complexity:
    - Best/Average: O(1)
    - Worst: O(n) with pathological hash collisions
    """
    return index.get(order_id)


def sort_orders_by_delivery_time(orders: Iterable[Order], descending: bool =
False) -> List[Order]:
    """Sort orders by delivery time.

    Complexity (Timsort):
    - Best: O(n)
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    return sorted(orders, key=lambda order: order.delivery_time,
reverse=descending)
```

```python
def sort_orders_by_price(orders: Iterable[Order], descending: bool = False) ->
List[Order]:
    """Sort orders by price.

    Complexity (Timsort):
    - Best: O(n)
    - Average/Worst: O(n log n)
    - Space: O(n)
    """
    return sorted(orders, key=lambda order: order.price, reverse=descending)


def print_recommendation_table(rows: List[Tuple[str, str, str]]) -> None:
    """Print operation-algorithm-justification table."""
    headers = ("Operation", "Recommended Algorithm", "Justification")
    widths = [len(h) for h in headers]

    for row in rows:
        for i, value in enumerate(row):
            widths[i] = max(widths[i], len(value))

    separator = "-+-".join("-" * width for width in widths)
    print(" | ".join(headers[i].ljust(widths[i]) for i in range(3)))
    print(separator)
    for row in rows:
        print(" | ".join(row[i].ljust(widths[i]) for i in range(3)))


if __name__ == "__main__":
    sample_orders = [
        Order("O1001", "Spice Hub", datetime(2026, 3, 24, 13, 15), 420.0,
"Preparing"),
        Order("O1002", "Green Bowl", datetime(2026, 3, 24, 12, 45), 280.0,
"Delivered"),
        Order("O1003", "Pizza Point", datetime(2026, 3, 24, 13, 5), 650.0,
"Out for Delivery"),
        Order("O1004", "Burger Barn", datetime(2026, 3, 24, 12, 50), 310.0,
"Delivered"),
        Order("O1005", "Spice Hub", datetime(2026, 3, 24, 13, 35), 520.0,
"Placed"),
    ]

    print("\nAlgorithm Recommendation Table:\n")
    print_recommendation_table(algorithm_recommendations())

    order_index = build_order_index(sample_orders)

    print("\nSearch Tests:")
    print("Order O1003 ->", search_order_by_id(order_index, "O1003"))
```

```
    print("Order O9999 ->", search_order_by_id(order_index, "O9999"))

    print("\nSorted by Delivery Time (earliest first):")
    for order in sort_orders_by_delivery_time(sample_orders):
        print(order)

    print("\nSorted by Price (highest first):")
    for order in sort_orders_by_price(sample_orders, descending=True):
        print(order)
```

Output:

```
Algorithm Recommendation Table:

Operation              | Recommended Algorithm                      |
Justification
------------------------
-------------------------------------------------------------------------------------------
Search by order ID     | Hash table (dict index)                    | Average
O(1) lookup for unique IDs; best fit for high-frequency real-time queries.
Sort by delivery time  | Timsort (Python sorted) with datetime key  | Stable O(n
log n) worst-case and fast on real-world partially ordered data.
Sort by price          | Timsort (Python sorted) with numeric key   |
Production-grade stable sorting for billing and analytics views.

Search Tests:
Order O1003 -> Order(order_id='O1003', restaurant_name='Pizza Point',
delivery_time=datetime.datetime(2026, 3, 24, 13, 5), price=650.0,
order_status='Out for Delivery')
Order O9999 -> None

Sorted by Delivery Time (earliest first):
Order(order_id='O1002', restaurant_name='Green Bowl',
delivery_time=datetime.datetime(2026, 3, 24, 12, 45), price=280.0,
order_status='Delivered')
Order(order_id='O1004', restaurant_name='Burger Barn',
delivery_time=datetime.datetime(2026, 3, 24, 12, 50), price=310.0,
order_status='Delivered')
Order(order_id='O1003', restaurant_name='Pizza Point',
delivery_time=datetime.datetime(2026, 3, 24, 13, 5), price=650.0,
order_status='Out for Delivery')
Order(order_id='O1001', restaurant_name='Spice Hub',
delivery_time=datetime.datetime(2026, 3, 24, 13, 15), price=420.0,
order_status='Preparing')
```

```
Order(order_id='O1005', restaurant_name='Spice Hub',
delivery_time=datetime.datetime(2026, 3, 24, 13, 35), price=520.0,
order_status='Placed')

Sorted by Price (highest first):
Order(order_id='O1003', restaurant_name='Pizza Point',
delivery_time=datetime.datetime(2026, 3, 24, 13, 5), price=650.0,
order_status='Out for Delivery')
Order(order_id='O1005', restaurant_name='Spice Hub',
delivery_time=datetime.datetime(2026, 3, 24, 13, 35), price=520.0,
order_status='Placed')
Order(order_id='O1001', restaurant_name='Spice Hub',
delivery_time=datetime.datetime(2026, 3, 24, 13, 15), price=420.0,
order_status='Preparing')
Order(order_id='O1004', restaurant_name='Burger Barn',
delivery_time=datetime.datetime(2026, 3, 24, 12, 50), price=310.0,
order_status='Delivered')
Order(order_id='O1002', restaurant_name='Green Bowl',
delivery_time=datetime.datetime(2026, 3, 24, 12, 45), price=280.0,
order_status='Delivered')
```